BLOOM ∞

EVERY SUCCESSOR MUST EARN ITS PROOF.

Verified capacity. Conditional value.

A preregistered study of selective operator construction, causal artifact handoffs, and the measured operational cost of qualified succession.



Fresh task. Fresh proof. Explicit authority.

AGI ALPHA AGENT | TRANSFORMATIONAL INTELLIGENCE ∞

SUCCESSOR Ω / Local Harness integration

Vincent Boucher · MONTREAL.AI

23 October 2026, Montréal / Independent research edition 1

Research program: *The seed that invents its economy.*

This edition establishes a bounded engineering result; competitive and commercial targets remain evidence-gated.

Abstract

Software teams repeatedly ask which applications a dependency change can affect and what coverage work should follow. Recomputing every answer wastes work; trusting a portable result after its conditions change can produce a confident error. We implement BLOOM∞, a local system that constructs a graph-specific impact operator, verifies its exact domain, and passes its answers to a distinct minimum-cost coverage-planning successor. The system uses unchanged published AGI ALPHA / SUCCESSOR Ω runtime components, with inspectable native history and inactive restoration. Before confirmation, we freeze source, analysis and a balanced 48-world study; an independently executed 24-world shifted-size set and a complete deterministic replay follow. All 2,016 eligible confirmatory trials resolve exactly, all 288 input/authority refusals are correct, and all 144 conventional replacement artifacts are accepted. Removing receiver-proof checks causes 48/48 adversarial trials to accept incorrect outcomes. The performance verdict in the shared study runner is narrower: CPU point estimates improve by 2.9% against an expert-style policy and 4.2% against reverse indexing, but neither establishes the prespecified 10% advantage. Artifact retention reduces receiver CPU by 22.2% at the point estimate; its Bonferroni-adjusted percentile interval supports 19.4–24.9%, narrowly missing the required 20% lower bound. Fresh reproduction also misses the practical thresholds. A real public-manifest demonstration covers 1,136 nodes and 2,000 edges through seven native stages. Cash revenue is zero; complete monetary cost and demand are unmeasured. The contribution is an exact, portable and reproducible engineering system with prospective causal evidence and explicit limits, not a new reachability algorithm or proven autonomous commerce.

The measured verdict

Earned	Exact supplied-graph outcomes; qualified artifact transport; correctly bounded restoration; reproducible local execution.
Not established	The declared 10% competitive advantage; the declared 20% minimum receiver gain; full-cost commercial viability.
Useful today	Inspect public dependency impact and a finite coverage plan, export the evidence, and restore its history without inheriting authority.

Evidence identity. Freeze manifest: `ecd3df27...1116bcfbe`; 48 confirmatory worlds; 24 fresh reproduction worlds; 2,544 deterministic replay records. Full identities and commands appear in Section 10.

1 The capacity that survives a completed task

An accepted output can be useful in two ways. It may answer the present question, and it may supply a capability that changes the work required by a later question. The second value is conditional. Construction has a cost. Verification has a cost. A changed graph, a revised policy, or withdrawn permission can invalidate the receiving use. A successor must therefore earn its own proof.

This research starts with a practical, nonpersonal need in the project's public software: determine which application roots depend on a changed package, then produce an inspectable coverage plan for those roots. The first task is graph reachability. The second is finite weighted coverage. The successor reads an actual predecessor-produced impact artifact; it does not receive a fictional mission label or a presumed financial return.

TRANSFORMATIONAL INTELLIGENCE supplies the motivating question: can changing the problem-solving setup make a purpose easier to achieve without weakening that purpose? The transformation here is bounded and concrete. Repeated traversal can be replaced by a graph-specialized data program with independently checked applicability. Its usefulness depends on the workload and on the cost of keeping its proof current. The foundations distinguish this structural question from a claim of novel unrestricted planning or universal superiority [1–3].

The strongest ordinary method remains eligible. BLOOM may use direct traversal, memoization, component propagation, reverse indexing or finite Warshall closure. The conventional references receive the same choices, information and qualification criteria. Building something new is a conditional option, never a requirement for an impressive-looking trace. In the real-manifest demonstration, the expert-style policy selects conventional reverse indexing.

1.1 Contribution statement

This edition contributes four connected artifacts. First, it implements a source-bound portable impact operator with exact certificates and a separately checked coverage successor. Second, it contributes a timestamped, preregistered comparison that distinguishes competitive advantage from the causal benefit of retaining an artifact. Third, it exercises the published Alpha dispatcher, real native history, backup and restoration in a bounded local journey. Fourth, it delivers complete source, raw observations, independent checks, a public-manifest example and an inspectable evidence browser.

The verification propositions collect sufficient conditions for the implemented certificates. They are engineering arguments built from established graph techniques, not claims that SCC decomposition, dynamic programming, caching, algorithm selection or test coverage were invented here. Earlier project editions develop mission-relative capability inheritance and construction of a next capability [4, 5]. This edition starts its own problem, generator and comparative design, and adds controlled retention, replacement, corruption and resource interventions. No learned model constructs a new algorithm during the confirmatory run. The implemented construction specializes a maintained operator grammar to a particular graph; every resulting program remains finite and inspectable.

1.2 Three distinct meanings of success

Engineering success means exact results within the stated contract, readable failures, reproducible commands, preserved evidence and honest authority. **Scientific advantage** means passing the declared comparative thresholds against strong allowed alternatives. **Economic success** would additionally require supported demand and complete attributable costs. The first can be delivered while the latter two remain unproven. The paper keeps all three verdicts visible.

Research question. Under changing workload and resource constraints, does constructing and carrying a verified graph-specific operator improve complete operational cost over equally equipped conventional methods, and does its artifact causally reduce the work of a distinct successor? The answer is tested with declared practical thresholds; useful exactness alone cannot answer the comparative question.

2 Problem, operators, and qualified succession

The operational principle of BLOOM∞ is that every successor must earn its proof. In this study, the predecessor supplies a reusable dependency-impact operator; the successor uses its answers to construct a finite coverage plan. These are distinct tasks connected by consumed data. Neither a lineage label nor successful execution of the predecessor establishes the successor's validity.

Let $G = (V, E)$ be a finite directed graph, with $u \to v$ meaning that v depends on u. Let $A = (a_0, \ldots, a_{k-1})$ be the ordered application list. For a changed-node set $S \subseteq V$, the impact target is

$$I_G(S) = \{a \in A : \exists s \in S,\ s \rightsquigarrow_G a\}, \tag{1}$$

where reachability includes paths of length zero. A node therefore affects itself when it is an application. The ordering of A defines bit positions and is part of the artifact binding. We write $R(v)$ for the bitmask encoding $I_G(\{v\})$; a multi-source answer is the bitwise union of its source masks.

The successor receives tests $\mathcal{T}$, coverage sets $C_t \subseteq A$, and strictly positive integer weights c_t. For an impact set I, it computes

$$J^*(I) \in \arg\min_{J \subseteq \mathcal{T}} \left\{ \sum_{t \in J} c_t : I \subseteq \bigcup_{t \in J} C_t \right\}. \tag{2}$$

Equal-cost solutions use the lexicographically smallest tuple of sorted test identifiers. Empty impact produces the empty feasible plan; uncovered requirements produce an explicit infeasible result. The implementation bounds cases to ten applications and twelve tests; the synthetic instances use six applications and ten tests, permitting exact finite optimization. Weights are declared synthetic durations, not measured test runtimes or money. Coverage of these requirements does not establish preservation of every fault detectable by the original test suite [6, 7].

The schema `bloom-impact-operator/v1` stores a strategy, ordered applications, rows, certificate, and world, policy, environment and source digests. It declares `authority: none`. A compiled row index is a data-specialized operator executed by a fixed interpreter. Its graph-specific contents are constructed during the run; the underlying algorithms and interpreter are maintained software. This is finite specialization, not discovery of a new graph algorithm or unconstrained program synthesis.

2.1 Certificates establish exactness, not merely closure

There are two complete-index certificates. Both are checked against the supplied graph rather than a producer-provided edge summary. Range, dimension, partition and integer-type checks precede the semantic checks. In particular, a content digest alone supplies no reachability proof.

Proposition 1: component recurrence. Suppose a certificate partitions V into nonempty sets $\mathcal{P}$; every set induces a strongly connected subgraph; and every edge between distinct sets advances in a supplied topological order. For $C \in \mathcal{P}$, let L_C encode the applications inside C. Suppose the verifier establishes

$$B_C = L_C \vee \bigvee_{(C,D) \in E_{\mathcal{P}}} B_D \quad \text{and} \quad R(v) = B_C \quad (v \in C), \tag{3}$$

where $E_{\mathcal{P}}$ is reconstructed from E. Then every row is exactly the reachable-application mask of its node.

Proof. Forward and reverse traversal within a proposed component, from one representative, establish that every member reaches every other member. The checked topological order makes the quotient graph acyclic. A path from any member of C to an application either stays in C or first exits along an edge to a successor component. Conversely, strong connectivity lets every member reach the tail of each such exit. Thus the reachable applications satisfy precisely Equation 3. Reverse topological induction gives its unique solution, beginning with sinks, whose masks contain only their local applications. Equality of every node row with its component mask completes the argument. This verifies both absence of spurious applications and inclusion of every reachable application. □

The implementation constructs components by two graph traversals and propagates application masks over the condensation. The independent verifier checks the partition, induced strong connectivity, topological edges and recurrence afresh. For the bounded application count, structural verification is linear in graph and certificate size; larger masks add their explicit bit-operation cost. Strong-component computation and transitive closure are established techniques [8, 9].

Proposition 2: reverse-path witnesses. Alternatively, suppose each application owns its own bit. For every claimed bit $a \in R(v)$ with $v \neq a$, the certificate supplies an actual edge $v \to p_a(v)$, the same claimed bit at $p_a(v)$, and nonnegative integer ranks satisfying $r_a(v) > r_a(p_a(v))$. The application root points to itself with rank zero. Unclaimed bits have absent witnesses. Finally, every graph edge $u \to v$ satisfies $R(v) \subseteq R(u)$. Then the rows encode exactly all reachable applications.

Proof. For a claimed bit, following its witness strictly decreases a nonnegative rank. It cannot continue indefinitely or cycle; a nonroot cannot terminate at rank zero because its required successor would have a smaller nonnegative rank. The chain therefore reaches the corresponding application through actual edges. There are no false positives. For the opposite direction, a path $v = v_0 \to \cdots \to v_\ell = a$ starts from the application's required ownership bit. Repeated edge closure propagates that bit backward to every v_i, including v. There are no false negatives. □

Reverse traversal from each application constructs these witnesses alongside the index. The verifier performs $O(k|V| + |E|)$ structural checks, with arithmetic cost determined by encoded integers and masks. Ordinary edge closure without the descending witnesses would accept an all-ones overapproximation; witnesses supply the missing inclusion in the true reachability relation. Memoized operators instead advertise partial rows, each replayed against independent BFS. Every memoized and direct output also receives fresh BFS checking, including cached outputs.

2.2 Construction and conventional alternatives

The candidate and expert selection policies can choose direct traversal, memoization, component propagation, reverse-per-application indexing and bitset Warshall closure. The second reference deliberately follows a fixed reverse-index strategy, with memoization then direct traversal as its legal fallbacks. It has the same information and wrapper, while the full-palette expert provides the primary equally equipped competitor. All receive the complete query batch, current graph, applications, test catalogue, available constructors and payload cap. A shared wrapper charges construction attempts, checks actual canonical serialized bytes, and records fallback to an available memoized or direct operator. The cap is not a claim about resident memory.

The candidate uses frozen graph-work estimates and requires at least 20% estimated headroom before preferring component or Warshall specialization over its available conventional alternatives. This threshold is a policy assumption, not an observed CPU saving. The estimates approximate graph work; they do not price all certificate, integration, maintenance or failure-risk costs. The policy therefore approximates the binding

computational constraint rather than solving a complete expected-value optimization. The measured end-to-end test is deliberately able to reject its usefulness. An independently written expert-style policy uses its own transparent workload estimates over the same palette. A conventional reverse-index policy is the second reference. The constructors and qualification boundary are shared deliberately: the comparison tests the policies within this implementation envelope, not differently optimized reimplementations of the same primitives. No claim is made that the reference is the globally fastest existing package. Practical reachability work already demonstrates the importance of initialization, update and query tradeoffs [10].

3 Prospective comparison and artifact interventions

Competitive advantage and artifact dependence are different estimands. For world w, let $T_{B,w}$ and $T_{C,w}$ denote complete measured operational CPU for candidate and conventional policies. The competitive ratio is

$$\rho_C = \exp\left(\frac{1}{W}\sum_{w=1}^{W}\log\frac{T_{B,w}}{T_{C,w}}\right). \tag{4}$$

Both policies have the same artifact and construction opportunities. Separately, let $S_w(K)$ be receiver CPU with the qualified predecessor artifact retained, and $S_w(\varnothing)$ receiver CPU when it is withheld and reconstruction remains allowed. Their paired ratio tests saved receiver work. Total predecessor-plus-receiver CPU is also reported. An artifact-deprived receiver is an intervention arm, not the principal competitive baseline. This explicit separation follows the logic of defining causal treatments before inspecting outcomes [11].

The protocol reserves twelve development worlds, at most two versioned development cycles, 48 confirmatory worlds and 24 fresh reproduction worlds. Confirmation crosses four topologies, four node counts and three query-batch sizes. Reproduction crosses the same topologies and query counts with two shifted sizes. The graph world is the independent unit; repeated queries and derived stages are not additional independent observations. The generator and seed ranges are known to the authors. Confirmation uses seeds 20001–20048; reproduction uses 30001–30024. The topologies are sparse DAG, layered DAG, hub graph and cyclic modules; node counts are 48, 72, 96 and 128 in confirmation and 64 and 160 in reproduction, with query batches of 8, 32 and 96. This is untouched execution under a disclosed generator, not a concealed external benchmark. Protocol, source and analysis were frozen and durably persisted before confirmation, preserving the distinction between declared tests and later explanation [12].

Stable and withheld arms use three complete timed replicates with reset method-local state and randomized execution order. The direct-walk no-construction arm and the proof-disabled corruption diagnostic each use one execution per world. Other declared interventions use one execution: conventional replacement, graph change, rehashed semantic corruption, policy change, authority revocation, scarce storage, constructor outage, withholding combined with outage, malformed input, infeasible coverage, and incomplete observed dependencies. A constructor outage does not invalidate a previously constructed data operator; it restricts new construction. The policy-change case removes a test from the eligible catalogue. Corruption modifies a supplied mask and recomputes its content hash, testing semantics rather than accidental checksum failure. Interventions occur before receiver execution.

Five gates separate reproducibility, quality, competition, succession and economics. Deterministic records must replay exactly, excluding timing and host-dependent metadata. Every eligible complete-graph task must resolve correctly, including explicit infeasibility; authority and malformed-input refusals are evaluated separately. The primary competitive endpoint uses only stable authorized worlds and the median of three complete CPU observations. It includes validation, policy choice, attempted construction, operational qualification, transport, revalidation and successor checking. The native seven-stage orchestration is measured separately in the practical demonstration, not inserted into every scientific kernel trial. Thus these

primary times cover the full declared runner operation, not the full product stack. A separately measured research audit uses independent BFS and exhaustive subset enumeration; it is excluded symmetrically from method latency and included in research cost.

Competitive success requires a simultaneous upper ratio bound below 0.90 against both conventional references without loss of correctness or completion. Artifact benefit requires a receiver ratio bound below 0.80 at equal quality. The three contrasts form one family: 10,000 fixed-seed world-bootstrap replicates [13] produce 98.333333% two-sided percentile intervals, providing nominal Bonferroni family coverage of 95%. Bootstrap coverage is approximate. A subgroup or inner-kernel improvement cannot replace the primary verdict. Scientific execution is capped at 1,200 CPU seconds and 256 MiB of new evidence; interrupted campaigns retain incomplete denominators and failed attempts.

3.1 Boundaries of independence and proof

The checker was authored separately within the same research session. Its BFS, finite subset enumeration, and certificate checks do not import candidate kernels. This reduces shared implementation error but is neither organizational independence nor a blinded external evaluation. Reproduction in another process establishes reproducibility under recorded conditions, not universal portability.

World, policy, proof and authority remain separate predicates. The receiver serializes and reopens the artifact, checks current graph/application, policy and source/environment bindings, and rechecks semantics. Restored history has no active proof or grant. Revoked receiver authority requires refusal even after legitimate predecessor production. Missing edges illustrate the observability boundary: exactness over an incomplete supplied graph can still omit real impact. The intervention reports that limitation instead of treating a valid hash as evidence of graph completeness.

The project stages are a trace through these operations: Identify binds the need; Out-Learn reads declared structure; Out-Think compares eligible operators; Out-Design constructs only when justified; Out-Strategise binds constraints; Out-Execute qualifies and computes; Out-Evolve evaluates the distinct coverage successor and retains or terminates its history. These labels do not enlarge the native runtime's authorization or make a changed plan inherit an earlier proof.

4 Relation to established work

Instance-conditioned operator choice is algorithm selection [14]; parameter and procedure tuning have extensive configuration literature [15]. Reusable program libraries and construction of later solutions from earlier abstractions are also established [16]. Here, the contribution sought is controlled evidence about when a qualified, portable operator pays for its complete measured operational cost and enables a successor under explicit constraints. The familiar machinery is identified rather than renamed as a discovery.

Coverage reduction has a longstanding test-to-requirement formulation [7]. Weighted greedy set cover provides an important scalable conventional alternative [17]; this experiment uses exact optimization only because its catalogue is deliberately small and grants the same optimizer to every policy. Safe regression selection has stronger assumptions and goals than coverage preservation [6]. Consequently, no synthetic planning weight is reported as measured testing saved or preserved fault detection. Finally, portable experiment packaging and provenance precede this work [18]. Exportability preserves inspectable evidence; semantic applicability and authority must still be established at the receiving boundary.

4.1 What is new, and what remains established engineering

Programming by examples can synthesize transformations within a specified grammar [19]; DreamCoder extends this with learned reusable abstractions [16]. BLOOM's confirmatory operator grammar and construction algorithms are fixed in advance. Its new per-world object is graph-specific data plus a certificate. It does not reproduce those systems' learned program discovery, and calling this finite construction "invention" does not establish equivalent capability.

Workflow planning already reasons about ordered task decomposition and applicability [20]. Reuse of qualified components has deep engineering roots [21]; content-addressed deployment and provenance support inspectable portability [18, 22]. This edition's additional evidence is the explicit, prospective separation of a fair selection comparison from artifact-retention, replacement and proof-ablation interventions, connected to the project's actual native lifecycle. Its contribution is that integrated experiment and usable bounded implementation, not ownership of the underlying planning or reuse idea.

Adaptive experimentation can invalidate ordinary holdout interpretation when the same evaluation answers guide repeated selection [23]. Here, twelve development worlds support two visible development cycles; no confirmatory outcome is used to revise the policy. The fresh-size set remains separately reported after the original failure. The resulting procedure is a simple frozen-split design, not a new reusable-holdout method. The paper's negative comparative result is part of the contribution.

5 Results: exact outcomes, unmet advantage thresholds

The confirmatory campaign contains 48 independent graph worlds and 2,544 trial executions. The fresh reproduction contains 24 worlds and 1,272 trials. A further 2,544 executions replay the original confirmation; they add computational cost, not independent worlds. All planned trial keys are present. Raw records, case files, operator artifacts, replacement receipts and the frozen analysis are retained, including the diagnostic failures.

5.1 The decision tree resolves without a superiority claim

Table 1 gives the prespecified decision tree and its final disposition. Failure of a later gate does not erase an earlier engineering result. It prevents the stronger associated claim. The original analysis correctly leaves G1 pending until the independent replay is available; a separate content-hashed review receipt closes it. The original analysis file is not rewritten to manufacture contemporaneous certainty.

Gate	Pass condition	Verdict	Consequence
G1	Exact deterministic replay under frozen identities	**PASS**	All 2,544 records, 48 cases and 480 operators match.
G2	Every eligible resolution exact; expected refusals and artifact boundaries correct	**PASS**	Retain the bounded supplied-graph claim.
G3	Upper CPU ratio bound < 0.90 against both conventional references	**FAIL**	No demonstrated 10% competitive advantage.
G4	Upper retained/withheld receiver ratio bound < 0.80, at equal quality	**FAIL**	Artifact effect is measured; required minimum gain is not established.
G5	Supported demand and complete comparative costs	**INCOMPLETE**	No commercial-viability or self-financing claim.

Table 1: Prespecified gates, final evidence and failure dispositions. G1 is closed by the later independent reproduction receipt. G3 and G4 retain their failed verdicts after reproduction.

Every eligible complete-graph trial resolves exactly: 2,016/2,016 in confirmation and 1,008/1,008 in fresh reproduction. These include correct infeasibility, not just successful plans. All 288 and 144 respective malformed-input or authority refusals are correct. All 1,008 and 504 required stale-or-withheld artifact rejections occur, and every conventional replacement is accepted: 144/144 and 72/72. These are execution counts within clustered worlds. Descriptive 95% Wilson intervals for the proportion of worlds with all eligible outcomes correct are $[0.926, 1]$ over 48 worlds and $[0.862, 1]$ over 24. They do not prove a zero production failure rate.

5.2 The practical performance hypotheses fail

Table 2 and Figure 1 report all three declared contrasts. The intervals use the same multiplicity adjustment specified before confirmation. The candidate's 2.86% and 4.21% point improvements are smaller than the required 10%. The interval against the expert policy also includes no improvement. The interval against reverse indexing excludes parity on this run, but excludes the required 10% saving as well. A statistically distinguishable small difference is not the declared practical success. A descriptive hindsight oracle, selecting the faster conventional median separately in each world, averages 4.0731 ms; BLOOM/oracle geometric ratio is 0.9983. On fresh reproduction, the oracle averages 4.4977 ms and the ratio is 1.0378. This predeclared descriptive lower bound is computed in supplementary reporting from frozen arrays, carries no new hypothesis test, and is not an executable policy: its selection information and cost are unavailable prospectively.

Set	CPU ratio	Estimate	Adjusted interval	Required upper
Confirmation	BLOOM / expert	0.9714	[0.9375, 1.0051]	< 0.90
48 worlds	BLOOM / reverse	0.9579	[0.9156, 0.9979]	< 0.90
	Retained / withheld receiver	0.7780	[0.7508, 0.8059]	< 0.80
Reproduction	BLOOM / expert	1.0079	[0.9653, 1.0725]	< 0.90
24 fresh worlds	BLOOM / reverse	0.9608	[0.8651, 1.0483]	< 0.90
	Retained / withheld receiver	0.8051	[0.7639, 0.8552]	< 0.80

Table 2: All prespecified primary contrasts. Ratios below one favor BLOOM or retention. Each interval has nominal 98.333333% coverage; the three-contrast family has nominal 95% Bonferroni coverage. Percentile-bootstrap coverage is approximate. No row passes its practical threshold.

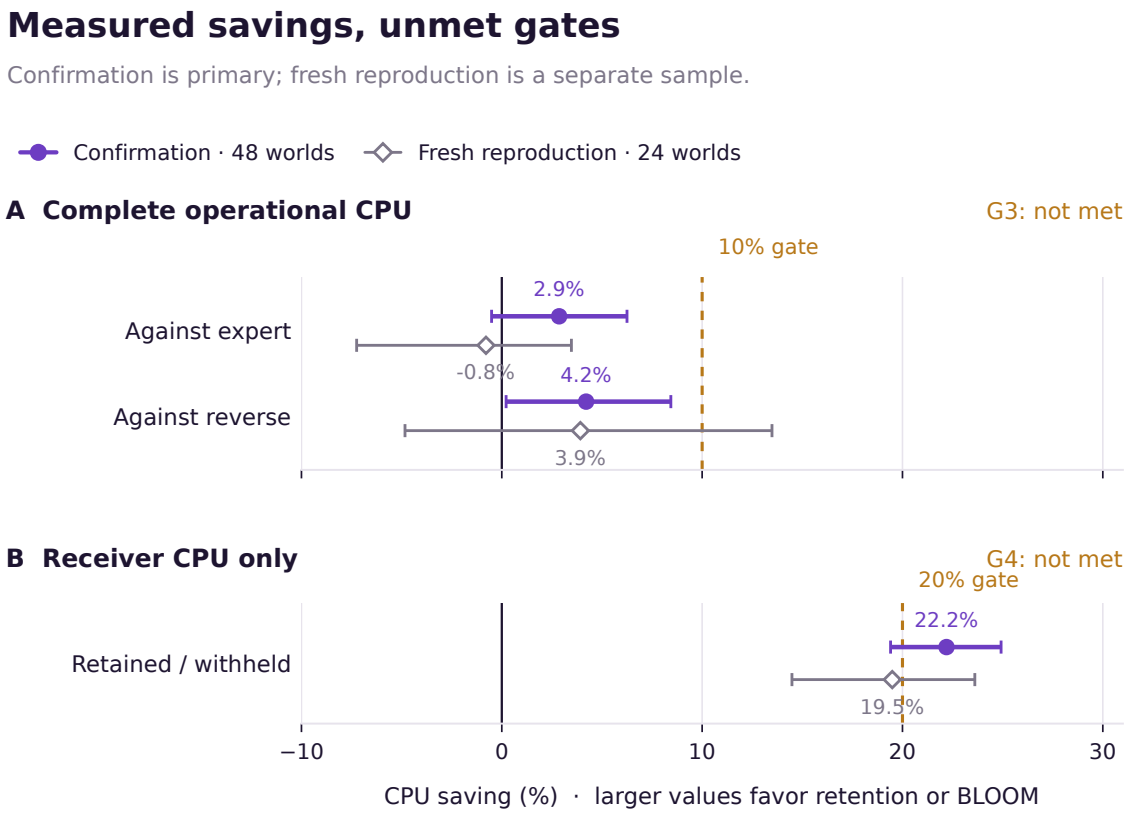


Figure 1: Effect estimates against the actual decision thresholds. Confirmation and fresh reproduction remain separate; pooling them after a failure would change the declared test. Source: frozen analyses, with the later independent G1 receipt shown only as provenance.

The arithmetic mean of the per-world median operational CPU is 4.0863 ms for BLOOM, 4.1929 ms for the expert policy, and 4.2568 ms for reverse indexing in confirmation. These means describe absolute cost; the geometric paired ratios answer the primary relative question. On fresh shifted-size worlds the corresponding means are 4.6993, 4.6306 and 4.9114 ms. The expert policy is then faster than BLOOM at the point estimate. Small Python workloads, interpreter overhead and host scheduling limit the portability of these timing magnitudes.

Both BLOOM and the expert choose SCC propagation on all 48 stable confirmatory worlds and all 24 reproduction worlds. There is no evidence here that the candidate identifies a superior standard-case operator. Its small timing differences arise within the same shared construction and verification pipeline, with different policy work and execution noise. The reverse baseline uses reverse indexing throughout. Figure 4 shows where constraints actually change behavior: payload scarcity and lack of an available constructor require ordinary fallbacks. The experiment did not create an artificially weak comparator to produce an invention win.

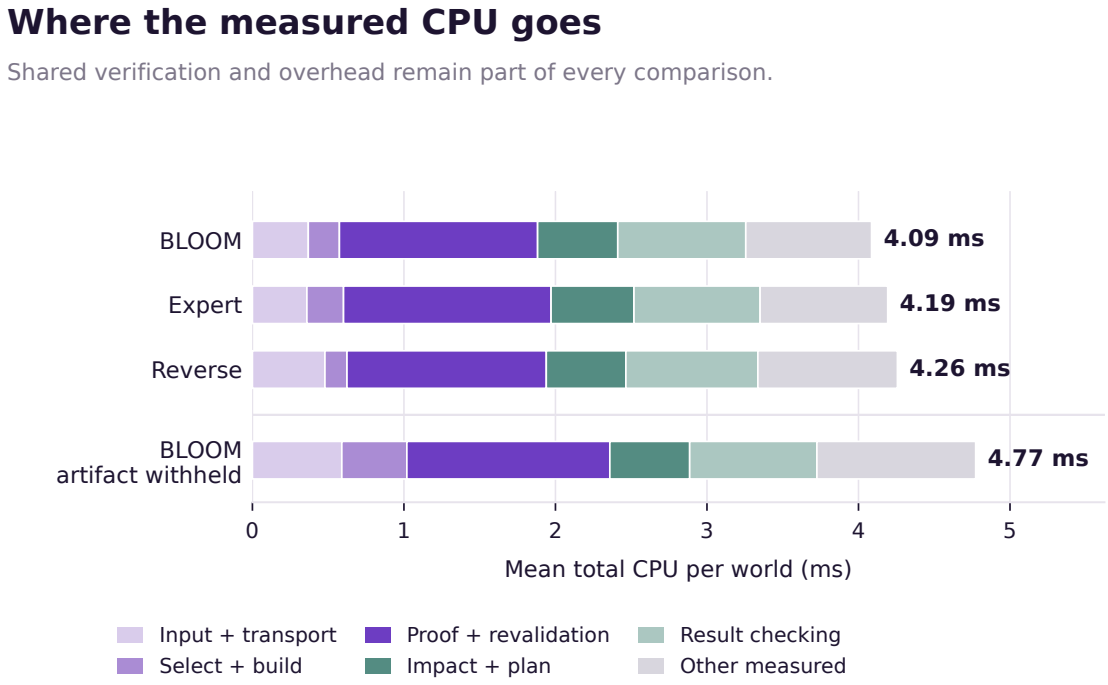


Figure 2: Complete operational cost by phase. Each world contributes its observed median-total-CPU replicate; all phase values come from that same replicate. Means therefore add exactly to the reported total, including the measured residual. Withholding adds receiver reconstruction while predecessor creation remains charged. This is an absolute-cost description, not an additional hypothesis test.

5.3 A useful artifact is not an irreplaceable artifact

The retained/withheld intervention estimates a controlled benefit of carrying a verified operator into the receiver. Retention improves receiver CPU by 22.20% at the confirmatory point estimate. Its adjusted saving interval is 19.41–24.92%; the required lower bound was 20%. The point estimate crosses the goal while the interval does not. G4 therefore fails. Fresh reproduction yields a 19.49% point saving with a 14.48–23.61% interval, also below the declared criterion.

The dependency itself is concrete: the coverage successor consumes the predecessor's impact artifact and its digest. Holding the graph and query fixed while removing that artifact forces reconstructive work. But successful conventional replacement in every declared replacement trial shows that the producer's identity is not necessary. The useful object is a qualified capability with the correct contents and bindings. We claim neither exclusive transfer nor a monopoly on successor creation.

These interventions are prospective controlled executions under a synthetic generator. They provide stronger evidence than retrospectively deleting a cost column from one successful trace. They still do not establish market demand, dependence in a deployed organization, or necessity when all alternative information and construction are allowed. Replacement construction is measured separately, once per actual creation, and attributed to each relevant counterfactual arm without being added repeatedly to the cash or research ledger.

6 Causal diagnosis and robustness

The study separates three mechanisms that can easily be confused: selecting an operator, constructing its graph-specific data, and qualifying its use at a receiving boundary. The stable selection comparison offers no novel-selector advantage because the strongest policies choose the same operator. The no-construction arm answers a narrower question: what happens if the same workload always uses direct traversal? All 48 confirmatory direct arms remain exact, with mean total CPU 4.8393 ms. This descriptive contrast suggests that specialization can save work in this finite setting. It is not the primary fair-competition test, and its single execution per world does not replace the three-replicate endpoint.

Verification has a distinct causal role. The rehashed-corruption treatment alters a real application mask and recomputes the content digest. BLOOM rejects and rebuilds all 48/48 confirmatory corrupted artifacts; all three policies do so in 144/144 verified corruption trials. In the deliberately unsafe diagnostic, disabling only receiver proof leads to 48/48 false acceptances and incorrect outcomes; fresh reproduction repeats this on 24/24 worlds. Checksums, lineage, and successful predecessor execution do not substitute for semantic revalidation. This diagnostic is not an offered operating mode.

6.1 When exactness is insufficient

Every method can be exact relative to a supplied dependency graph and still miss actual impact if that graph omits a real dependency. The deliberate missing-edge challenge removes incoming edges to a target application after constructing a complete hidden world, and forces the first query to node zero. The frozen protocol prose says “removes an edge”, whereas the frozen implementation removes all incoming edges to that application. This wording mismatch is a disclosed deviation, not a random single-edge failure model; the challenge remains outside G2 and the primary timing tests. All methods omit true affected applications on 1,657 of 2,176 query instances in confirmation and 897 of 1,088 in reproduction. All 48 and 24 respective worlds exhibit omissions. These counts describe a designed stress condition, with correlated queries inside worlds; they are not prevalence estimates for public package manifests.

This is the most consequential failure mode for deployment. Better graph algorithms cannot infer undeclared runtime dependencies from a checksum. The public example is therefore an inventory of declared package relationships. It is not a certificate that an application will pass its real tests after a dependency update. Acquiring and validating representative runtime or build dependencies is a separate measurement problem.

6.2 Changed constraints produce bounded behavior

A graph change or policy revision invalidates the transported binding and causes fresh qualification. The policy intervention removes an actually eligible test, so the successor’s feasible set changes. Authority withdrawal after legitimate predecessor production refuses the receiving execution. A 2 KiB payload ceiling exercises failed construction and a charged conventional fallback. Constructor unavailability leaves already-constructed data usable; combining outage with withholding forces an available traversal-based replacement. Malformed inputs are rejected and uncovered requirements terminate as infeasible. None of these conditions licenses an indefinite repair loop.

Correct resolution has several meanings

Confirmation · world counts, with boundary and failure arms kept visible

	BLOOM	Expert	Reverse
Stable handoff	48 / 48	48 / 48	48 / 48
Artifact withheld	48 / 48	48 / 48	48 / 48
Equivalent replacement	48 / 48	48 / 48	48 / 48
Graph changed	48 / 48	48 / 48	48 / 48
Policy changed	48 / 48	48 / 48	48 / 48
Artifact tampered	48 / 48	48 / 48	48 / 48
Storage cap: 2 KiB	48 / 48	48 / 48	48 / 48
Constructor unavailable	48 / 48	48 / 48	48 / 48
Withheld + unavailable	48 / 48	48 / 48	48 / 48
Cover infeasible	48 / 48	48 / 48	48 / 48
Authority revoked	48 / 48	48 / 48	48 / 48
Malformed graph	48 / 48	48 / 48	48 / 48
Hidden dependency edges	**48 / 48**	**48 / 48**	**48 / 48**
Tampered · proof bypass	**48 / 48**	not run	not run

Qualified plans
Correct infeasibility
Correct refusal
Incomplete input: omissions
Bypass: wrong output
Not run

48 worlds per cell; stable/withheld repeats are collapsed within each world.
Incomplete graphs: 1,657/2,176 query outputs omit a true impact per method.
That query count is descriptive and clustered. Supplied-graph correctness cannot certify missing facts.

Figure 3: Robustness includes correct refusals and visible failures. An infeasible world contains at least one correctly infeasible query and may also contain feasible ones. Missing edges are evaluated against the complete hidden graph, outside the supplied-graph guarantee. The proof-disabled corruption arm is deliberately unsafe. Source: complete confirmatory records, not selected successful traces.

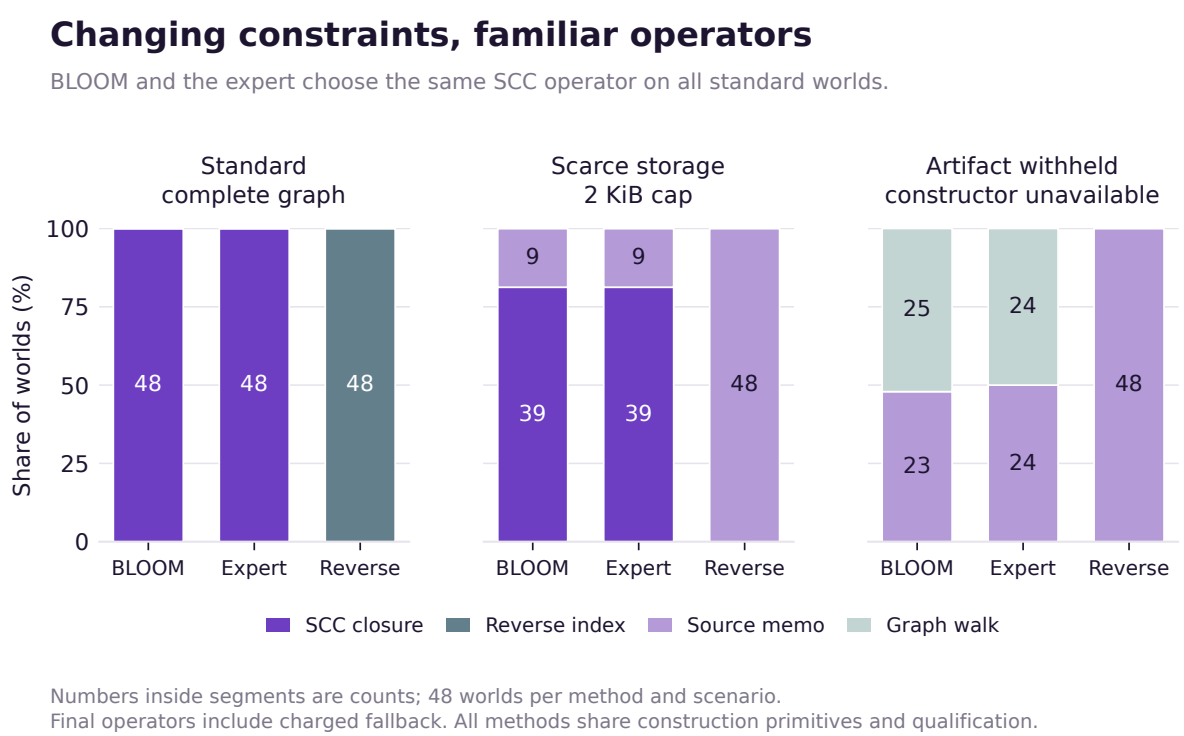


Figure 4: Actual strategy choices. The same strong SCC choice dominates stable conditions for BLOOM and expert. Constraints make conventional fallback visible. All methods have the same allowed palette and payload checks; a rejected build is charged and recorded. Counts are worlds, not claims of independent stages.

6.3 Failure severity and what was not measured

The implemented boundary distinguishes an incorrect accepted answer, an unnecessary refusal, a correct refusal, correct infeasibility, and incomplete observed world data. In the authorized complete-graph arms there are no incorrect accepted outcomes or unexpected refusals. Semantic corruption without receiver proof is a severe false acceptance. Incomplete-world omission is a severe external-validity limitation even when the supplied-graph proof passes. Neither category is hidden inside an average quality score.

Memory evidence is also limited. Canonical artifact bytes and the process high-water RSS are recorded; a serialized payload cap does not measure per-method resident memory. CPU and monotonic wall time are measured separately. Counter events such as traversed edges, row checks and subset scores are not interchangeable resources and are never summed into a fictional price or detector acquisition count.

7 Economics: conditional value, incomplete full cost

The research records no cash revenue, firm offer, independently supported demand, or external customer acceptance. No financial transaction or new paid infrastructure purchase was performed. Those observations do not make authoring, execution or maintenance free. Human effort, model/service authoring, inherited hardware and integration maintenance have unknown attributable fees. G5 is therefore incomplete.

The scientific campaign ledger sums each completed campaign receipt once: 47.8877 CPU seconds across preserved development, confirmation, fresh reproduction and exact replay. External correctness audits account for 5.3831 seconds within that total, not in addition to it. Five distinct measured native integration runs separately total 6.1983 CPU seconds, including the 1.1277-second public demonstration. Three initial development calls add 0.1802 seconds. The resulting documented-compute subtotal is 54.2661 seconds.

The protocol prose said native overhead was included in campaign cost; the campaign receipts actually exclude it. The final ledger explicitly includes that separate subtotal rather than rewriting the original receipts. Figure generation, document production, unmetered authoring and other integration preparation are outside that measured campaign total and remain explicit unpriced work. The ledger does not present this documented subtotal as the complete cost of producing the research.

Quantity	Evidence and interpretation	Classification
Revenue / firm offers	Zero recorded; no transaction or offer validation	Observed absence
CPU / wall time	Per phase, trial and completed campaign; audit subsets reconcile	Measured resources
Detector acquisitions	Zero; no physical detector or exposure experiment	Observed count
Coverage weights	Positive synthetic test durations; no real tests run	Modeling input
CPU tariff	0.02, 0.10, 1.00 USD per core-hour; assumed for sensitivity	Hypothetical price
Extra engineering	0, 1, 8, 40, 160 hours at an assumed 100 USD/hour	Sensitivity only
Human, API, hardware, maintenance	Attributable fees and effort incomplete	Unknown material cost
Opportunity cost	Compare the same accepted workload under the strongest tested conventional policy	Conditional alternative

Table 3: Economics keeps unlike quantities separate. An assumption is not a quoted price; test weights are not cash savings; omitted fees prevent a complete margin claim.

For policy m, a conditional full-value model is

$$N_m(n) = R_m(n) - F_m - n\,(p\,t_m/3600 + u_m)\,, \tag{5}$$

where R_m is supported revenue if any, F_m attributable fixed cost, t_m measured CPU seconds per accepted workload, p an assumed CPU-hour price, and u_m other per-use cost. In this study R_m is not established and material parts of F_m and u_m are unknown. For equal accepted service, the appropriate alternative is the measured expert policy; its work is available within the same resource envelope. A score-based "opportunity cost" unrelated to an alternative use is not introduced.

The confirmation mean difference against that expert is only 0.0001066 CPU seconds per world. At the illustrative tariff of 0.10 USD per core-hour, it is approximately 0.003 USD per million uses. Recovering even one extra assumed 100 USD engineering hour would require about 33.8 billion uses if that small saving were real, stable and transferable. Fresh reproduction reverses the CPU point difference against the expert. This arithmetic argues against a commercial-saving claim from the present workload. It is a sensitivity bound, not a revenue forecast or evidence that such usage exists.

A larger payoff might come from a validated reduction in expensive testing, analysis time or integration mistakes. Those quantities were not measured. Coverage-plan weights cannot be promoted into observed avoided test cost. The successor's output is counted once; neither hypothetical future revenue nor reuse of an already-paid artifact creates additional cash in the ledger. A 100-to-10,000 monetary trajectory is not tested or assumed here. The protocol also mentioned a value-per-use sensitivity grid; the frozen analysis produced CPU-price and engineering-hour sensitivities only. The omitted grid is a disclosed reporting limitation, not a missing positive result.

8 The executable journey and the project foundations

The thin integration uses actual published Alpha runtime code and preserves its lifecycle and authority boundaries [24–26]. It does not install four complete products or claim a cross-repository deployment.

Eight unchanged upstream source files, their license and a source manifest are bundled; source digests are checked before and after installed callbacks. The public example runs the native dispatcher and records a valid journal, followed by an inactive restore.

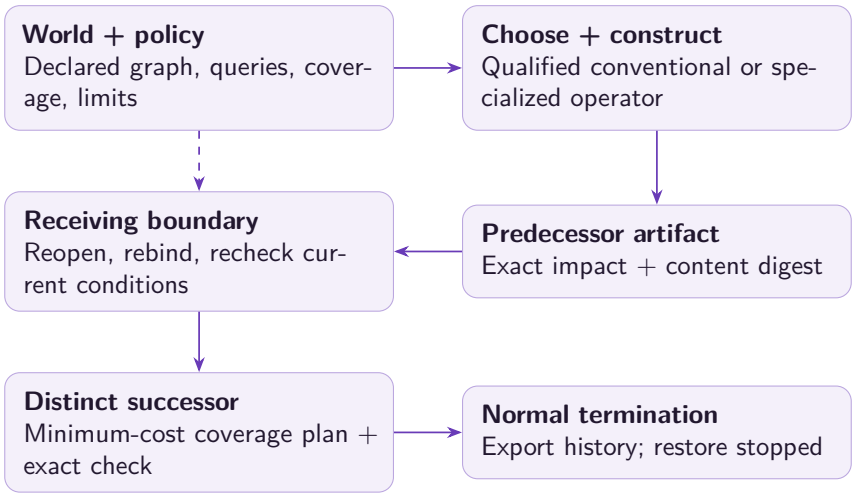


Figure 5: The actual dependency is consumed data. Current world, policy, source/environment and authority conditions must hold at the receiver. The dashed edge supplies those current conditions. An intact history does not bypass the receiving boundary.

8.1 Seven connected stages, concrete inputs and outputs

Table 4 gives the executed public-demo journey. Native lifecycle family names are exact code values; the correspondence is a local allocation of responsibilities rather than a claim that the names are synonymous APIs. Every stage has a callback, input, returned artifact and explicit check. No transaction is manufactured to make the sequence look complete.

Journey	Native family	Concrete work / output	Qualification
Identify	`constitution`	Bind objective and supplied case	Finite local scope, authority ceiling
Out-Learn	`evidence`	Read declared dependency structure	Input schema and graph validity
Out-Think	`challenge`	Compare allowed operator work estimates	Same information and palette
Out-Design	`formation`	Construct graph-bound data operator	Payload cap; recorded fallback
Out-Strategise	`admission`	Bind proof, limits and current conditions	Independent exact certificate
Out-Execute	`verification`	Compute impact artifact	Independent answer check
Out-Evolve	`renewal`	Consume impact; produce coverage plan	Exact finite optimum; stop

Table 4: Actual public-demo callbacks and native allocations. A future workload with no useful successor should stop after its accepted output; the architecture does not require seven ceremonial operations.

The adapter calls the native `SuccessorStore` operations `create_institution` and `register_release`. Its `JobDispatcher` calls are `seal`, `dispatch`, `execute_tool` and `complete` [27, 28]. It also exercises `SuccessorStore.recover` and `Journal.verify`, `backup` and `restore`. The demonstration has seven completed stages and 48 journal events. Public deterministic fixture signing material is used solely for the local test identities. The declared execution mode is `synthetic-local`, with `authority: none`; it supplies no external credential or transaction authority.

The restored record is stopped and globally paused, with zero active proofs and grants. This is a successful restoration of inspectable history, not an automatically runnable successor. The user can inspect and export the record; a new execution still needs applicable artifacts, environment, policy and authorization. An additional schema adapter can produce genuine unsigned project record shapes, but packaging a post-study receipt does not move the earlier scientific preregistration timestamp.

8.2 Four repositories: verified contribution and remaining gap

Repository	Verified responsibility and contribution	Actual scope / missing integration
AGI-Alpha-Agent-v0	SUCCESSOR records, local dispatcher, journal, backup/restore; foundations and publication surfaces	Native bounded workflow executed. Narrow source snapshot, not full product installation.
AGIJobManager	Discovery request interface and Manager USDC accounting route	Source-audited. New graph task not admitted through its installed finite receiver.
AGIJobsv0	Discovery request interface and AGIALPHA jobs route	Source-audited. No external job submission, auction or settlement.
AGI-Fleet-Linux	Execution-side boundaries and restore qualification metadata	Source-audited; external execution remains unprovisioned, restore qualification `NOT_RUN`.

Table 5: Additive use of the actual foundations, without converting interface inspection into a claim of deployed integration. Exact repository commits and source receipts ship with the bundle.

Source inspection of Manager, Jobs and Fleet grounds the distinctions in Table 5 [29–31]. The shared discovery schema is `successor-discovery-work-request/v1`. Its exact resource fields are `acquisitions`, `search`, `checking`, `memoryBytes`, `storageBytes`, `workerCapacity`, `installations` and `elapsedMs`. Binding fields include `missionId`, `revision`, `purposeDigest`, `charterDigest`, `modelDigest`, `sourceDigest` and `evaluatorDigest`. Manager's route is `manager-usdc` with six-decimal USDC; Jobs uses `jobs-agialpha` with eighteen-decimal AGIALPHA. The bounded fixtures declare `capBaseUnits: "0"` and `DRAFT_NOT_SUBMITTED`. Current installed receivers execute their finite `inspectCampaign` operation; they do not thereby admit this new graph task. That adapter and qualification work remains missing.

World, policy, proof and authority each answer a different question. *World* binds the declared graph and application order. *Policy* binds coverage eligibility and limits. *Proof* establishes the relevant semantic claim for the current source/environment. *Authority* determines whether the present execution is allowed. A valid answer in an earlier world does not supply any of the other three. Existing held Fleet, Harness and richer-site operations remain held.

8.3 Five Alpha contracts: source behavior, not performed transactions

The canonical source names are `AscensionAccess`, `NovaSeed`, `AscensionRiskOracle`, `AscensionJobMarket`, and `AscensionMark` [32]. Their behavior constrains any eventual integration (Table 6). This edition performs no EVM experiment: the required local toolchain dependencies were absent. The bundle records `EVM_NOT_RUN`. Source inspection is not a contract-test pass.

MARK phases are `None`, `Funding`, `Bloomed`, `Closed`. The job market states are `None`, `Auction`, `Assigned`, `Submitted`, `Paid`, `Failed`, `Refunded`. Routing checks the committed Merkle leaf and index, budget, current authority and time horizon. A successor that changes the committed plan cannot silently mutate the old seed. Parent lineage is provenance, not permission, proof or finance.

Implemented AGIALPHA token behavior covers deposits and payouts, an eighteen-decimal token, a 1%

Contract	Implemented behavior	Boundary for BLOOM
AscensionAccess	Business, Agent and Validator eligibility combine live ENS and expiring governor admission	A local graph proof grants no contract role.
NovaSeed	Transferable ERC-721 genome commits capsule, plan, evidence, parent and job count; no genome/plan setter	A changed immutable Merkle plan needs a fresh seed and explicit authority.
AscensionRiskOracle	Stake-backed eligible-validator review, bounded quorum and capacity reservation; green status rechecks eligibility	Risk review is not proof of scientific correctness, demand or profit.
AscensionJobMarket	Auction, assignment, submitted evidence, validator voting, payouts and terminal refunds	No auction or settlement is exercised. Missing review quorum can end refunded.
AscensionMark	Funded, unexpired, currently green and eligible seed can move from Funding to Bloomed	`bloom` changes phase, sets execution horizon and emits; it spawns no successor and creates no wealth.

Table 6: Verified source responsibilities. The claim is limited to the inspected commits and implementation; no local or external contract execution is reported.

gross-payout burn and a 5% validator allocation split between approving validators. These mechanics establish neither a token price nor economic advantage. Manager USDC accounting is a separate route and unit. Proposed incentives or future self-financing mechanisms remain speculative and are not entered as study revenue.

9 A useful local demonstration

The supplied public fixture parses pinned npm lockfiles from Alpha's web client and Manager. It contains 1,136 package-location nodes, 2,000 dependency edges and two application roots. The parser resolves Node-style ancestor locations and local links; an independent acceptance program checks 2,016 declarations/link records. Sixteen unresolved optional peers are disclosed. The larger Jobs lockfile is preserved as source evidence but excluded from this finite demonstration envelope.

Before the demonstration, a reader has static package declarations but no qualified impact index or coverage plan. Afterwards, the reader can choose one of 17 fixed change queries, inspect affected application roots, see the chosen operator and exact qualification, inspect the coverage successor, and export the evidence. For example, a change to `alpha-web::node_modules/vite` reaches `alpha-web::<root>`; the corresponding synthetic plan is `synthetic-alpha` with weight 7. A Babel-parser path through Babel core and the React plugin gives another inspectable dependency chain. These are declared graph relations, not measurements of tests saved.

The default expert policy chooses conventional reverse indexing for this larger public graph. It qualifies through root ownership, real-edge descending path witnesses and graph-edge closure. All 17 answers match independent acceptance. The complete seven-stage native run takes 1.1277 CPU seconds and 1.1277 wall seconds on the recorded host, including orchestration, checks, journal and restore. This value is deliberately not compared as though it were the same workload as the millisecond synthetic kernels.

A short walkthrough

1. Run. From the extracted study directory, use Python 3.12 with the pinned integration dependencies, then run the public-case command below.
2. Inspect. Open the emitted `index.html`. Choose a changed package. The report shows the affected roots, why the conventional operator qualifies, and the coverage plan that consumes its impact.
3. Follow the proof. Read `summary.json`, the operator, impact and successor artifacts, then the native journal and inactive restoration receipt.
4. Stop normally. No deployment, test execution, payment or additional mission is required. The exported record remains available for a later explicitly authorized use.

```
python3 -m pip install -r integration/requirements.txt
python3 demo.py --case public_case/case.json \
  --method expert --output runs/public-demo
```

The setup step obtains two pinned Python packages and their declared dependencies; the demo itself uses local data and performs no external submission. With prerequisites already installed, the second command is the one-command demonstration. `python3 demo.py -output runs/bloom-demo` uses the deterministic development fixture instead. An output directory must be fresh so prior evidence is not overwritten.

Users can safely inspect declared dependency impact and finite coverage choices today. They should not use this study alone to suppress real regression tests, approve a deployment, authorize spending or certify a dependency inventory complete. Production use requires representative observed dependencies, measured test coverage and runtime, independently validated correctness, appropriate organization-owned authority, and operational maintenance. None is created by an attractive interface.

10 Reproducibility, integrity and recovery

The source-and-protocol freeze was timestamped 2026-10-24 02:02:15 UTC and durably persisted before confirmatory generation and execution. It was then recorded in coordination issue 4850. The complete SHA-256 identity of `protocol/FREEZE.json` is

`ecd3df2706d034d17a1441c62fd4dba0c7c9`
`2f5ea4d22327011bf5e1116bcfbe`

The 23 frozen source identities cover the candidate, generator, independent checker, runner, analysis, demo and native integration. Development has two preserved versions and twelve allowed seeds. The first development smoke attempts retain an honest limited console record; complete return values were not retained, a disclosed retention deviation; the later 636 development trial records are complete. No post-confirmation source, analysis, threshold or seed change was used to seek a passing result.

The independently authored checker uses separate BFS and exhaustive subset logic. Its author executes the fresh reproduction before reviewing the original performance statistics, then replays the original deterministic records. All frozen source hashes remain unchanged. The replay excludes timing and host-dependent fields from the equality claim; CPU timing is expected to vary. The reproduction is a different process and authoring role in the same research session, not another institution or concealed benchmark.

The scientific core requires only Python's standard library. The recorded interpreter is CPython 3.12.14. The local native demonstration additionally uses Pydantic 2.14.0 and cryptography 50.0.2. Plotting uses

the recorded scientific-Python versions and a fixed generator; every plotted series has a CSV and source digest. The complete archive includes the authoritative PDF, LaTeX, bibliography, vector figure sources, code, protocol, all raw campaigns, analysis, source receipts and an evidence browser. The manifest hashes every included member except itself; the archive digest is recorded externally to avoid a circular self-hash.

```
python3 run_study.py --split confirmation \
  --output results/local-confirmation
python3 analyze.py results/local-confirmation
python3 run_study.py --split reproduction \
  --output results/local-reproduction
python3 analyze.py results/local-reproduction
python3 public_case/acceptance.py --check
```

These commands produce a new reproduction, not additional preregistered evidence for the original claim. Keep the archived receipts and use new named output directories under `results/`; the cumulative-budget check scans `development/` and `results/`. The runner refuses reused output directories. Each rerun adds recorded work without adding new independent worlds. The recorded scientific budget is 1,200 CPU seconds and 256 MiB of new evidence. Bounded local experiments, including failed attempts and all intended exclusions, remain inspectable.

The evidence browser maps major claims to their file, JSON pointer, classification and digest. Its 72 unique scientific worlds are kept distinct from repeated queries, stage counts and replayed executions. The archival checkpoint was reopened and its exact contents verified before confirmation. Final publication assets have independent file digests and a recovery manifest. A standalone document service may change PDF metadata; the exact canonical publication bytes remain inside the verified complete archive. Publication status is established only by the actual repository review and served-asset verification receipt, not by a prospective URL in this paper.

10.1 Repository and runtime identities

Source	Pinned Git commit
AGI-Alpha-Agent-v0	`dfc79c4ae81ac612cbff9865a0af8389b6ff3de9`
AGIJobsv0	`9983478d42c32fe399f28dd9b0e0173f904b8a11`
AGIJobManager	`c5958eae872458c2f3cac48fea0b4bbfadc6ac6c`
AGI-Fleet-Linux	`45759fb096faf8988f58ae5ea71248d0f35a97c7`

Table 7: Exact source states inspected and, for the narrow native Alpha snapshot, executed. Later upstream changes do not silently update the scientific artifact.

11 Limitations and the next decisive test

The finite generator favors small exact graphs and coverage catalogues. Four topologies and shifted sizes exercise controlled variation, not representative software distributions. Authors know the generator and seed ranges. Shared constructors keep access fair but limit conclusions to this implementation, rather than the fastest compiled graph library. The standard candidate and expert choose the same operator, so selector novelty is unproven. The experiment does not test online learned synthesis or unrestricted tool invention.

Proof applies to supplied structure. Deliberately missing edges expose the boundary sharply. Positive synthetic coverage weights are not measured test runtimes, and requirement coverage does not establish fault preservation. The live project demonstration validates parsing, data flow, exact supplied-graph answers and local lifecycle behavior; it is not independent customer validation. The source-only interfaces for Manager, Jobs, Fleet and the contracts have not become an integrated external execution path.

Resource measurement is substantial but incomplete. CPU, wall time, artifact bytes and discrete computational events are recorded; per-method energy, monetary fees, human effort, full resident-memory attribution and maintenance are not. Same-session independent code and execution reduce some implementation risk without replacing external replication. Bootstrap intervals are approximate, and the all-correct sample cannot exclude rare production failures.

The most valuable next study is a preregistered evaluation on unseen public software revisions with observed build dependencies and measured real test coverage and runtime. A conventional regression-selection implementation should remain equally eligible, with human integration time charged. That study would test whether a qualified impact artifact saves an expensive real workflow rather than microseconds of synthetic computation. It should proceed as a new version with new untouched cases and an explicit authority boundary, retaining this edition's failed hypotheses.

12 Conclusion

BLOOM∞ delivers a finite capability that survives export, earns renewed proof, and supplies a distinct coverage-planning successor through actual project runtime components. Exact outcomes, conventional replacement, honest restoration and reproducible evidence are established within the stated envelope. The prespecified competitive advantage, minimum receiver benefit and full-cost commercial viability are not established. The system remains useful because it preserves that distinction in its code, interface, ledger and publication.

Infinite potential is an architectural possibility: a useful accepted artifact may reveal another opportunity. This study executes one justified handoff and then stops. It offers no evidence of infinite growth. *Every successor must earn its proof.*

Research and artifact statement

This independent edition builds on the project's published foundations and native infrastructure. Earlier editions and their URLs retain their own identities. Research design, implementation, checking, plotting and manuscript preparation used AI assistance within one coordinated session; distinct checking roles are not claimed to be independent institutions. No external account credentials, security settings, financial commitments or external transactions were changed. Source licenses are preserved, with upstream files and their exact identities listed in the bundle. Operational permission and publication review remain separate from scientific evidence.

References

[1] Vincent Boucher. Transformational Intelligence: Learning to change the game. Research manuscript, MONTREAL.AI, October 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/Transformational_Intelligence.pdf. October 2026 edition; source and publication identities verified at the pinned repository revision.

[2] Vincent Boucher. Out-Execute: Constructing the conditions of completion. Revised research manuscript, MONTREAL.AI, October 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/Out_Execute.pdf. 21 October 2026. Cover subtitle: From repeated work to reusable means of execution.

[3] Vincent Boucher. Invent the Economy: Transformational intelligence for creating viable markets. Research manuscript, MONTREAL.AI, October 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/Invent_the_Economy.pdf. October 2026 edition. Both PDF and source paths verified in the pinned repository; this manuscript has no entry in the inspected publication index.

[4] MONTREAL.AI. Transformational Autonomous Enterprises: SUCCESSOR Ω harness—mission-relative revalidation for capability inheritance across four execution systems. Research and engineering edition Writer #4, research edition 01, MONTREAL.AI, October 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/papers/transformational-autonomous-enterprises-v1.pdf. 23 October 2026. Attributed to MONTREAL.AI and Writer #4 on the retained title page.

[5] MONTREAL.AI. The possibility foundry: Autonomous enterprises that invent their next capability. Research edition Writer #5, edition 01, MONTREAL.AI, October 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/papers/possibility-foundry-v1.pdf. 23 October 2026. Research direction: Vincent Boucher. Cover subtitle: From Autonomous Capability Synthesis to Measurable Expansion of Economic Opportunity.

[6] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997. doi: 10.1145/248233.248262. URL https://www.cs.purdue.edu/homes/xyzhang/spring07/Papers/p173-rothermel.pdf.

[7] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology*, 2(3):270–285, 1993. doi: 10.1145/152388.152391. URL https://www.cs.ucr.edu/~gupta/research/Publications/Comp/p270-harrold.pdf.

[8] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972. doi: 10.1137/0201010. URL https://epubs.siam.org/doi/10.1137/0201010.

[9] Stephen Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962. doi: 10.1145/321105.321107. URL https://dl.acm.org/doi/10.1145/321105.321107.

[10] Kathrin Hanauer, Monika Henzinger, and Christian Schulz. Faster fully dynamic transitive closure in practice. In *18th International Symposium on Experimental Algorithms*, volume 160 of *Leibniz International Proceedings in Informatics*, pages 14:1–14:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.SEA.2020.14. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SEA.2020.14.

[11] Donald B. Rubin. For objective causal inference, design trumps analysis. *The Annals of Applied Statistics*, 2(3): 808–840, 2008. doi: 10.1214/08-AOAS187. URL https://arxiv.org/abs/0811.1640.

[12] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114. URL https://www.pnas.org/doi/10.1073/pnas.1708274114.

[13] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552. URL https://projecteuclid.org/journals/annals-of-statistics/volume-7/issue-1/Bootstrap-Methods-Another-Look-at-the-Jackknife/10.1214/aos/1176344552.full.

[14] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Academic Press, 1976. doi: 10.1016/S0065-2458(08)60520-3. URL https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=1098&context=cstech.

[15] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization*, volume 6683 of *Lecture Notes in Computer Science*, pages 507–523. Springer, 2011. doi: 10.1007/978-3-642-25566-3_40. URL https://ml.informatik.uni-freiburg.de/wp-content/uploads/papers/11-LION5-SMAC.pdf.

[16] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, pages 835–850. ACM, 2021. doi: 10.1145/3453483.3454080. URL https://dspace.mit.edu/entities/publication/86d768ab-558f-4fb1-b92e-9288de643ccb.

[17] Václav Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3): 233–235, 1979. doi: 10.1287/moor.4.3.233. URL https://pubsonline.informs.org/doi/10.1287/moor.4.3.233.

[18] Fernando Chirigati, Dennis Shasha, and Juliana Freire. ReproZip: Using provenance to support computational reproducibility. In *5th USENIX Workshop on the Theory and Practice of Provenance*, TaPP '13. USENIX Association, 2013. URL https://www.usenix.org/conference/tapp13/technical-sessions/presentation/chirigati.

[19] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, pages 317–330. ACM, 2011. doi: 10.1145/1926385.1926423. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/popl11-synthesis.pdf.

[20] Dana S. Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Füsun Yaman. SHOP2: An HTN planning system. *Journal of Artificial Intelligence Research*, 20:379–404, 2003. doi: 10.1613/jair.1141. URL https://arxiv.org/abs/1106.4869.

[21] M. Douglas McIlroy. Mass produced software components. In Peter Naur and Brian Randell, editors, *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968*, pages 138–155. NATO Scientific Affairs Division, 1969. URL https://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF. Conference held in 1968; report published in January 1969.

[22] Eelco Dolstra, Merijn de Jonge, and Eelco Visser. Nix: A safe and policy-free system for software deployment. In *Proceedings of the 18th Large Installation System Administration Conference*, LISA '04, pages 79–92. USENIX Association, 2004. URL https://www.usenix.org/legacy/publications/library/proceedings/lisa04/tech/full_papers/dolstra/dolstra_html/index.html.

[23] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375. URL https://dwork.seas.harvard.edu/publications/preserving-statistical-validity-adaptive-data-analysis.

[24] MONTREAL.AI. AGI-Alpha-Agent-v0. Source repository, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/commit/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9. Pinned source snapshot inspected 24 October 2026.

[25] MONTREAL.AI. SUCCESSOR Ω: Architecture decision 001. Repository architecture document, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/docs/agent/SUCCESSOR_ARCHITECTURE.md. Source document reports implementation in release 1.24.0 with no external qualification or production grant; inspected at the pinned revision.

[26] MONTREAL.AI. SUCCESSOR Ω protocol data types. Python source module, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/alpha_factory_v1/core/runtime/successor/protocol.py. Exact definitions of WorldProgram, PolicyProgram, ProofProtocol, ProofReceipt, AuthorityEnvelope, and resource and mission boundaries.

[27] MONTREAL.AI. SUCCESSOR Ω native job dispatcher. Python source module, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/alpha_factory_v1/core/runtime/successor/jobs.py. Unchanged source bytes retained in the bounded local integration; source identity is recorded in integration/vendor/SOURCE.json.

[28] MONTREAL.AI. SUCCESSOR Ω state and history. Python source module, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/alpha_factory_v1/core/runtime/successor/state.py. Pinned implementation of native state, export, and restoration; restoration is not an inherited proof or authority grant.

[29] MONTREAL.AI. AGIJobManager: Discovery lab work-request receiver. JavaScript source module, 2026. URL https://github.com/MontrealAI/AGIJobManager/blob/c5958eae872458c2f3cac48fea0b4bbfadc6ac6c/scripts/integrations/discovery-lab-work-request.mjs. manager-usdc route; synthetic-local inspection; authority none; submission unsupported.

[30] MONTREAL.AI. AGIJobsv0: Discovery lab work-request receiver. JavaScript source module, 2026. URL https://github.com/MontrealAI/AGIJobsv0/blob/9983478d42c32fe399f28dd9b0e0173f904b8a11/integrations/alpha-agent/discovery-lab-work-request.mjs. jobs-agialpha route; synthetic-local inspection; authority none; submission unsupported.

[31] MONTREAL.AI. AGI-Fleet-Linux: SuccessorController. JavaScript source module, 2026. URL https://github.com/MontrealAI/AGI-Fleet-Linux/blob/45759fb096faf8988f58ae5ea71248d0f35a97c7/fleet/packages/AGI_Agent/successor/controller.mjs. Installed Controller–Studio–Companion–ComputeWorker path; fresh runtime, policy, ticket, principal, and confirmation checks remain required.

[32] MONTREAL.AI. AGI-Alpha-Agent-v0: Ascension contract sources. Solidity source modules, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/tree/dfc79c4ae81ac612cbff9865a0af8389b6ff3de9/contracts/ascension. Inspected files: AscensionAccess.sol, NovaSeed.sol, AscensionRiskOracle.sol, AscensionJobMarket.sol, and AscensionMark.sol. MARK bloom implements the Funding-to-Bloomed transition; it does not spawn successors or create wealth.